LOVELY PROFESSIONAL UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC TASK - 2

CSE-316

(Operating Systems)

PROJECT REPORT

IPC Debugger - Inter-Process Communication Analyzer

Submitted By:

Aditya Kumar Singh | Reg. No.: 12400959
Abhishek Singh | Reg. No.: 12406001
Venkateswar Reddy Boggeti | Reg. No.: 12403005

Submitted To:
Prof. Isha Mam
Department of Computer Science and Engineering

Transforming Education Transforming India

TABLE OF CONTENTS

1. Project Overview 3
1.1 Introduction 3
1.2 Objectives 3
1.3 Scope of the Project 3
2. Module-Wise Breakdown 4
2.1 Module 1: Backend C Server 4
2.2 Module 2: Frontend Dashboard 4
2.3 Module 3: Event Logging System 5
3. Functionalities 6
4. Technology Used 7
5. Flow Diagram 8
6. Revision Tracking on GitHub 9
7. Conclusion and Future Scope 10
8. References 11
Appendix A: AI-Generated Project Elaboration 12
Appendix B: Problem Statement 14
Appendix C: Solution / Code 15
Project Demonstration 16

1. Project Overview

1.1 Introduction

Inter-Process Communication (IPC) is a fundamental concept in modern operating systems that enables multiple processes to exchange data and synchronize their operations. Understanding and debugging IPC mechanisms is critical for developers working on concurrent, distributed, or system-level software applications. The IPC Debugger project was conceived to address the growing need for a real-time, visual, and interactive debugging environment that allows developers, students, and researchers to observe and analyze IPC behaviour without relying on low-level kernel tools or complex manual tracing techniques.

1.2 Objectives

The primary objectives of this project are as follows:

- To simulate the three core IPC mechanisms: Pipes, Message Queues, and Shared Memory within a controlled and observable environment.
- To develop a real-time web-based dashboard that visualizes live data flow, communication events, and system states across processes.
- To implement an intelligent event logging system that records all IPC activities with timestamps, event types, and process identifiers.
- To detect and report potential deadlock conditions and communication bottlenecks automatically.
- To provide an educational tool that bridges the gap between theoretical OS concepts and practical implementation.

1.3 Scope of the Project

The scope of this project encompasses the design, development, and deployment of a full-stack IPC debugging system. The backend is developed using the C programming language and handles all low-level IPC simulation and socket communication. The frontend is built with HTML5, CSS3, and JavaScript, and communicates with the backend server via HTTP to fetch and display live data. The project is scoped to operate on Linux-based systems and targets the simulation of three IPC

mechanisms. It does not cover kernel-level instrumentation or distributed system debugging across multiple physical machines.

2. Module-Wise Breakdown

The IPC Debugger system is logically divided into three principal modules, each responsible for a distinct layer of the application. These modules interact with each other in a layered architecture to ensure separation of concerns, maintainability, and scalability.

2.1 Module 1: Backend C Server

The Backend C Server forms the core computational engine of the IPC Debugger. It is entirely written in the C programming language and is responsible for all process-level operations, IPC mechanism simulation, and serving data to the frontend via a lightweight socket-based HTTP server.

Architecture and Design:

The backend is structured as a multi-threaded server process. Upon startup, it binds to a specified TCP port (default: 8080) and listens for incoming HTTP GET requests from the browser-based frontend. The server employs POSIX threads (pthreads) to handle multiple concurrent client connections and to run IPC simulations in parallel without blocking the main listening loop.

IPC Mechanism Simulation:

- Pipes: The server forks child processes and establishes unidirectional communication channels using anonymous pipes (pipe() system call). Data packets are written to and read from the pipe, and the bandwidth, latency, and transfer state are tracked.
- Message Queues: POSIX message queues (mq_open, mq_send, mq_receive) are used to simulate asynchronous message passing between simulated processes. The server monitors queue depth, message priority, and delivery status.
- Shared Memory: POSIX shared memory segments (shm_open, mmap) are allocated and mapped into the address space of multiple simulated processes. A semaphore-based mutex ensures safe concurrent access to the shared region.

Deadlock and Bottleneck Detection:

The backend implements a basic circular wait detection algorithm for the shared memory module. It maintains a resource allocation graph and checks for cycles at regular intervals. Communication bottlenecks are detected by monitoring queue saturation levels and pipe buffer utilization thresholds.

2.2 Module 2: Frontend Dashboard

The Frontend Dashboard provides the visual interface through which users interact with the IPC Debugger. It is developed using HTML5, CSS3, and JavaScript, and requires no external JavaScript framework, ensuring lightweight and fast rendering in any modern browser.

Dashboard Layout:

The dashboard is divided into several panels: a summary status panel showing the active IPC mechanisms and their current states; a real-time data flow visualization panel displaying animated diagrams of inter-process communication; a metrics panel showing bandwidth utilization, queue depth, and latency graphs; and an event log panel that streams incoming log entries.

Communication with Backend:

The frontend uses the Fetch API and XMLHttpRequest to poll the C backend server at configurable intervals (default: 500 milliseconds). JSON-formatted responses from the backend are parsed and rendered dynamically using DOM manipulation and the Chart.js library for graphical representations.

User Interaction:

The dashboard allows users to start, pause, and reset IPC simulations via control buttons. Users can select which IPC mechanism to monitor, adjust simulation parameters such as message size and frequency, and trigger manual events like deadlock scenarios for testing purposes.

2.3 Module 3: Event Logging System

The Event Logging System is an integrated subsystem that operates across both the backend and the frontend to provide a comprehensive, timestamped audit trail of all IPC activities during a debugging session.

Backend Logging:
On the backend, every IPC event (message sent, message received, shared memory write, pipe read, deadlock detected, etc.) is written to an in-memory ring buffer. Each log entry contains: a high-resolution POSIX timestamp (clock_gettime), the event type identifier, the source and destination process IDs, the data payload size, and the IPC mechanism type. The ring buffer is flushed and serialized into JSON format upon every frontend polling request.

Frontend Log Display:
The frontend renders the log entries in a scrollable table within the Event Log panel. Entries are color-coded by severity (informational, warning, error) and can be filtered by event type, process ID, or IPC mechanism. The log can be exported as a CSV file for offline analysis.

Log Persistence:
Logs are additionally written to a flat text file (ipc_debug.log) on the server-side for persistence across sessions and for post-mortem analysis.

3. Functionalities

The IPC Debugger system offers a comprehensive set of features that collectively constitute a robust debugging and analysis environment. The major functionalities are enumerated below:

3.1 IPC Mechanism Simulation

- Simulation of Pipes for unidirectional data transfer between parent and child processes.
- Simulation of POSIX Message Queues for asynchronous, priority-based inter-process message passing.
- Simulation of POSIX Shared Memory with semaphore-based mutual exclusion for concurrent memory access.

3.2 Real-Time Visualization

- Live sequence diagram rendering on the web dashboard showing message flows between process entities.
- Animated arrows and state indicators that update in real time as IPC events occur.
- Dynamic charts for bandwidth utilization, message queue depth, and average latency per mechanism.

3.3 Event Logging

- Timestamped logging of every IPC event with microsecond-level precision.
- In-browser real-time log streaming with colour-coded severity levels.
- CSV export of the complete event log for external analysis.
- Persistent server-side log file for session-independent record keeping.

3.4 Deadlock Detection

- Automated circular wait detection using a resource allocation graph maintained in the backend.
- Immediate visual alert on the dashboard when a deadlock condition is identified.
- Detailed deadlock report specifying the processes and resources involved in the cycle.

3.5 Bottleneck Analysis

- Monitoring of pipe buffer fill levels and alerting when buffers reach saturation thresholds.
- Detection of message queue overflow conditions and slow consumer patterns.
- Identification of shared memory contention hotspots through lock acquisition time measurement.

3.6 Dashboard Control Features

- Start, pause, and reset controls for each IPC simulation module independently.
- Configuration panel for adjusting simulation parameters: message size, frequency, number of processes.
- Manual trigger for injecting fault conditions such as process delays and resource starvation.
- Responsive web design compatible with desktop browsers (Chrome, Firefox, Edge).

4. Technology Used

4.1 Programming Languages

Language	Purpose	Version
C (ISO C11)	Backend server, IPC simulation, socket communication, deadlock detection	GCC 11.x
HTML5	Frontend dashboard structure and semantic layout	HTML5 Standard
CSS3	Styling, responsive layout, animations, and visual design of the dashboard	CSS3 Standard
JavaScript (ES6+)	Frontend logic, API polling, DOM manipulation, Chart.js integration	ES2020

4.2 Libraries and Tools

Library / Tool	Language	Purpose
POSIX Threads (pthreads)	C	Multi-threaded request handling and parallel IPC simulation
POSIX Message Queues (mqueue.h)	C	Message queue IPC mechanism simulation
POSIX Shared Memory (sys/mman.h)	C	Shared memory IPC mechanism simulation
POSIX Semaphores (semaphore.h)	C	Mutual exclusion for shared memory access
POSIX Sockets (sys/socket.h)	C	TCP socket server for HTTP communication with frontend
Chart.js (v3.x)	JavaScript	Rendering dynamic line charts and bar graphs on the dashboard
Fetch API	JavaScript	Asynchronous HTTP polling of the backend JSON endpoints

4.3 Other Tools

Tool	Purpose
GCC (GNU Compiler Collection)	Compilation of C source code with POSIX compliance flags
GNU Make	Build automation for the backend server components
Git	Version control and collaborative development
GitHub	Remote repository hosting and revision tracking
VS Code	Primary integrated development environment for all code
Linux (Ubuntu 20.04+)	Target operating system; required for POSIX IPC API support
Google Chrome	Primary browser for dashboard testing and demonstration
Eraser.io	Sequence diagram generation for project documentation

5. Flow Diagram

The sequence diagram below illustrates the complete communication flow of the IPC Debugger system from the moment a user initiates a visualization request through the frontend dashboard, to the final rendering of the interactive chart. The diagram captures all seven principal actors in the system: the User, Frontend Dashboard, HTTP Request handler, C Backend Server, IPC Simulation engine, Event Logging subsystem, JSON Response formatter, and Frontend Visualization renderer.

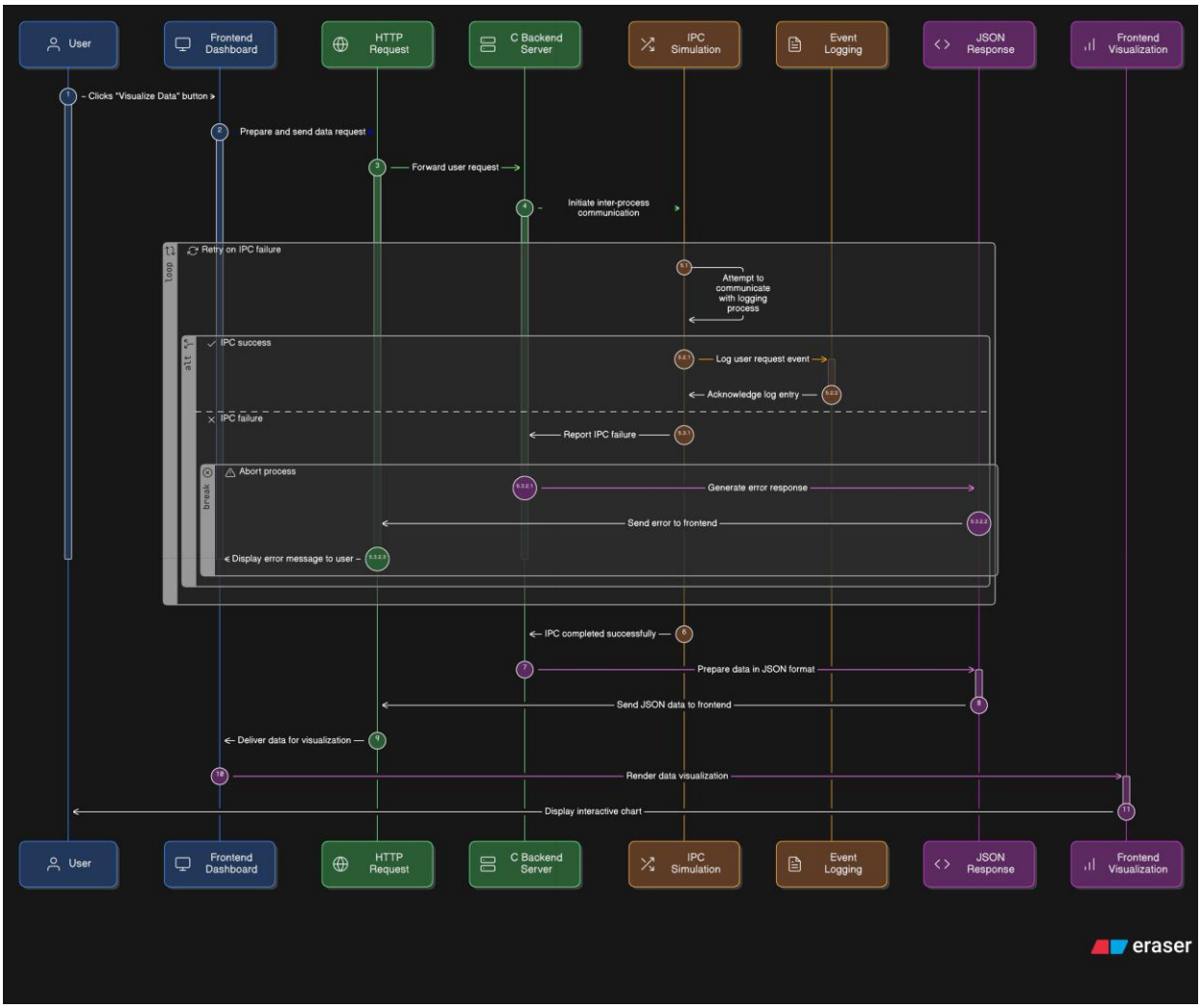

Figure 1: IPC Debugger System - Complete Sequence Diagram

5.1 Flow Description

The flow begins when the User clicks the 'Visualize Data' button on the Frontend Dashboard (Step 1). The dashboard prepares and dispatches an HTTP data request (Step 2), which is forwarded to the C Backend Server (Step 3). The backend server initiates inter-process communication with the IPC Simulation module (Step 4). Inside a retry loop (loop: Retry on IPC failure), the IPC Simulation attempts to communicate with the Event Logging process (Step 5).

If IPC succeeds (alt: IPC success), the Event Logging subsystem logs the user request event (Step 5.2.1) and acknowledges the log entry back (Step 5.2.2). If IPC fails (alt: IPC failure), a failure report is returned to the backend (Step 5.3.1). In the break condition (Abort process), the IPC Simulation generates an error response that is sent across to the JSON Response module (Step 5.3.2.1), forwarded to the frontend (Step 5.3.2.2), and displayed as an error message to the user (Step 5.3.2.3).

Upon successful IPC completion (Step 6), the C Backend Server prepares the result data in JSON format (Step 7) through the JSON Response module. The JSON data is sent to the frontend (Step 8) and delivered for visualization (Step 9). The Frontend Dashboard initiates a render data visualization call to the Frontend Visualization module (Step 10), which returns an interactive chart displayed to the User (Step 11).

6. Revision Tracking on GitHub

6.1 Repository Details

Attribute	Details
Repository Name	ipc-debugger-ca2
GitHub Link Repository Description Primary Language	https://github.com/abhii026/ipc-debugger-ca2 CA2 Project - IPC Debugger in C | BTech CSE C
Branch Strategy	main (production), dev (development)

6.2 Version Control Workflow

The team employed Git for systematic version control throughout the project lifecycle. All source code, documentation, and configuration files are maintained within the GitHub repository. The team followed a feature-branch workflow where each new module or feature was developed on a dedicated branch before being reviewed and merged into the main branch.

6.3 Commit Conventions

Commit messages followed a structured convention to ensure traceability and readability of the project history. The general format adopted was: [MODULE] Brief description of change. Representative commit categories include:

- [BACKEND] Implemented POSIX message queue simulation with priority handling
- [FRONTEND] Added real-time Chart.js integration for bandwidth visualization
- [LOGGING] Implemented ring buffer for in-memory event log management
- [DEADLOCK] Added circular wait detection algorithm in resource allocation graph
- [DOCS] Updated README with build instructions and project overview

6.4 Collaboration

GitHub Issues were used to track bugs and feature requests throughout development. Pull Requests were employed for code review before merging significant changes into the main branch, ensuring code quality and mutual review across all three team members.

7. Conclusion and Future Scope

7.1 Conclusion

The IPC Debugger - Inter-Process Communication Analyzer was successfully designed and implemented as a full-stack debugging environment that bridges low-level C-based operating system programming with a modern, interactive web-based user interface. The project demonstrates that complex IPC concepts such as Pipes, Message Queues, and Shared Memory can be made accessible and visually interpretable through thoughtful system design.

The backend C server reliably simulates the three IPC mechanisms using POSIX APIs and serves real-time data to the frontend through a lightweight socket-based HTTP server. The event logging system provides a granular, timestamped audit trail of all IPC activities, enabling accurate post-mortem analysis and real-time monitoring. The deadlock detection module demonstrates the practical application of resource allocation graph theory in a live system, while the bottleneck analysis features add operational value for performance-sensitive debugging scenarios.

From an academic perspective, this project reinforces the theoretical foundations of Operating Systems as taught in CSE-316, specifically the topics of inter-process communication, process synchronization, concurrency, and deadlock handling. It serves as a practical demonstration of how these concepts manifest in real software systems, making it a valuable educational artefact for the course.

7.2 Future Scope

Several avenues for future enhancement have been identified during the course of this project:

- Network IPC Support: Extending the simulator to include socket-based IPC across networked processes or machines, enabling debugging of distributed system communication patterns.

- Kernel-Level Tracing Integration: Integrating with Linux tracing frameworks such as eBPF or strace to capture actual kernel-level IPC events from real running processes, moving beyond simulation to genuine process monitoring.
- Persistent Session Management: Implementing a database backend (SQLite or PostgreSQL) to store and replay debugging sessions, allowing users to revisit historical IPC traces.
- Advanced Deadlock Resolution: Extending the deadlock detection module to include automated resolution strategies such as process preemption, resource reallocation, and rollback mechanisms.
- Cross-Platform Support: Porting the backend to support macOS (using Mach IPC primitives) and Windows (using named pipes and Windows shared memory APIs) for broader applicability.
- Machine Learning Anomaly Detection: Applying unsupervised machine learning algorithms to IPC event streams to automatically identify anomalous communication patterns indicative of race conditions or security vulnerabilities.
- Mobile Dashboard: Developing a responsive mobile version of the dashboard or a dedicated Android/iOS application for remote IPC monitoring.

8. References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[2] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed. Boston, MA: Addison-Wesley Professional, 2013.

[3] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook. San Francisco, CA: No Starch Press, 2010.

[4] R. Love, Linux System Programming: Talking Directly to the Kernel and C Library, 2nd ed. Sebastopol, CA: O'Reilly Media, 2013.

APPENDIX

Appendix A: AI-Generated Project Elaboration / Breakdown Report

The following elaboration was generated with the assistance of an AI language model (Claude by Anthropic) to provide a structured analytical breakdown of the project for academic documentation purposes.

A.1 Project Overview

The IPC Debugger is an integrated debugging platform designed to make Inter-Process Communication mechanisms transparent, observable, and analyzable in real time. The project addresses a significant gap in the existing developer toolchain: while tools like strace and ltrace can trace system calls at a low level, they do not provide an intuitive, visual, or interactive interface for understanding the flow and behaviour of IPC events over time. The IPC Debugger fills this gap by providing a simulated but architecturally faithful environment where all IPC events are captured, logged, and rendered visually on a web dashboard.

The system is architecturally divided into a C backend and a JavaScript/HTML frontend, connected over a local HTTP interface. The C backend handles all process-level simulation, event capture, and deadlock analysis, while the frontend provides the user-facing visualization and control interface. This separation of concerns ensures that the computationally intensive IPC simulation logic is isolated from the presentation layer, following best practices of layered software architecture.

A.2 Module Breakdown

Backend Module:

The backend is the most complex component of the system. It runs as a native Linux process, leveraging POSIX APIs for all IPC operations. The server is designed to be extensible, with each IPC mechanism implemented as a discrete simulation unit that can be enabled or disabled

independently. Thread safety is ensured through pthreads mutex locks on all shared data structures, including the event log ring buffer and the resource allocation graph used for deadlock detection.

Frontend Module:
The frontend dashboard is designed for clarity and usability. It employs a polling architecture rather than WebSockets to maintain simplicity and compatibility with the lightweight C HTTP server. The polling interval is configurable, allowing users to trade off between update frequency and network overhead. Chart.js is used for all graphical components, providing smooth animations and responsive rendering.

Event Logging Module:
The event logging subsystem is implemented as a cross-cutting concern that spans both backend and frontend. On the backend, the ring buffer implementation ensures that log entries are captured with minimal performance overhead. On the frontend, the log rendering component uses virtual scrolling techniques to handle large volumes of log entries without degrading browser performance.

A.3 Functionalities

The system provides end-to-end IPC debugging capabilities including simulation of all three major POSIX IPC mechanisms, real-time visual representation of message flows and state transitions, granular event logging with microsecond precision, automated deadlock detection based on resource allocation graph cycle detection, communication bottleneck identification through threshold-based alerting, and an interactive dashboard with full simulation lifecycle control.

A.4 Technology Recommendations

For future iterations, the following technology enhancements are recommended:

- Replace HTTP polling with WebSockets (using libwebsockets in C) for lower-latency real-time updates.
- Adopt React.js or Vue.js for the frontend to improve component reusability and state management.
- Integrate SQLite for persistent log storage to enable historical session replay.

- Use CMake as the build system for improved cross-platform compilation support.

A.5 Execution Plan

The project was executed in four structured phases. Phase 1 (Week 1-2) focused on requirement analysis, system design, and architecture documentation. Phase 2 (Week 3-4) covered the development of the C backend, including the IPC simulation modules, socket server, and event logging infrastructure. Phase 3 (Week 5-6) involved the development of the HTML/CSS/JavaScript frontend dashboard, Chart.js integration, and UI testing. Phase 4 (Week 7) was dedicated to integration testing, bug fixing, documentation preparation, and the recording of the project demonstration video.

Appendix B: Problem Statement

Modern operating systems rely heavily on Inter-Process Communication (IPC) mechanisms to enable co-operating processes to exchange data, synchronise their operations, and share system resources. The three primary IPC paradigms - Pipes, Message Queues, and Shared Memory - each exhibit distinct behavioural characteristics in terms of latency, throughput, synchronisation requirements, and potential failure modes. Debugging issues arising from incorrect IPC usage, such as deadlocks, race conditions, message loss, and communication bottlenecks, is a notoriously difficult task due to the concurrent, non-deterministic, and often kernel-mediated nature of these interactions.

Existing debugging tools such as strace, ltrace, and gdb provide system-call-level and instruction-level visibility but lack the ability to present IPC behaviour in an abstract, human-interpretable visual format. Developers working on concurrent or multi-process applications must therefore mentally reconstruct the sequence of IPC events from raw traces, a process that is time-consuming, error-prone, and inaccessible to students learning operating system concepts for the first time.

The problem this project seeks to address is: How can a software system be designed and implemented to simulate, monitor, visualize, and analyze Inter-Process Communication mechanisms in real time, providing developers and students with an intuitive, interactive, and instrumented debugging environment that makes IPC behaviour transparent, diagnosable, and educationally valuable?

This problem is significant because the absence of such a tool creates a barrier to understanding and debugging IPC-intensive software, particularly in educational settings where practical demonstration of theoretical concepts is essential for effective learning. The IPC Debugger project directly addresses this problem by delivering a full-stack solution that combines the performance and system-level access of C with the accessibility and visualisation capabilities of a modern web interface.

Appendix C: Solution / Code

Complete project source code is attached separately as per submission guidelines.

The full codebase is also available at the official GitHub repository:

`Repository: https://github.com/abhii026/ipc-debugger-ca2`

The repository contains the following primary source files and directories:

- server.c - Main C backend server with socket handling and HTTP response generation.
- ipc_pipe.c - Pipe simulation module implementing unidirectional process communication.
- ipc_msgqueue.c - Message queue simulation module using POSIX mqueue APIs.
- ipc_sharedmem.c - Shared memory simulation module with semaphore-based access control.
- event_logger.c - Ring buffer event logging system with JSON serialization.
- deadlock_detector.c - Resource allocation graph implementation and cycle detection algorithm.
- index.html - Frontend dashboard main page with Chart.js integration.
- dashboard.css - Stylesheet for the web dashboard UI.
- dashboard.js - JavaScript logic for API polling, data rendering, and user interactions.
- Makefile - Build automation script for the C backend components.
- README.md - Project documentation including build and run instructions.

Project Demonstration

A complete walkthrough of the IPC Debugger system has been recorded and made available on YouTube. The demonstration video covers the system architecture overview, startup and configuration of the C backend server, live simulation of all three IPC mechanisms via the web dashboard, real-time event log streaming, deadlock detection trigger and alert demonstration, and bottleneck analysis in action.

YouTube Video Link:
Link

GitHub Repo Link:
link